

Plan d'implémentation — Reproduction d'une approche auto-supervisée pour l'estimation de pose 2D/3D du cheval

Projet source : Sosa et al., *"A Horse with no Labels: Self-Supervised Horse Pose Estimation from Unlabelled Images and a Synthetic Prior"*, ICCVW 2023 **Contexte :** Stage d'été — Efrei Research Lab (distanciel) **Durée :** 8 semaines (2 mois) **Rôle du document :** Servir de feuille de route technique et méthodologique unique, du cadrage jusqu'à la livraison finale.

0. Required Inputs (informations à confirmer avant démarrage)

Le sujet fourni est précis sur la méthode et le planning, mais certains éléments opérationnels ne sont pas spécifiés. Ils sont listés ici plutôt qu'inventés.

#	Information manquante	Impact si non résolu	Qui doit trancher
1	Ressources de calcul allouées (GPU local, cluster Efrei, cloud, quota d'heures)	Détermine la taille de sous-ensemble d'entraînement réaliste (S6-S7)	Encadrant / IT Efrei
2	Accès effectif à TigDog (licence, lien de téléchargement à jour) — le sous-ensemble cheval de TigDog est parfois distribué de façon non centralisée	Bloque Semaine 1-2 si l'accès n'est pas validé tôt	Stagiaire + encadrant
3	Statut légal/licence des ~60 vidéos YouTube à extraire (droits d'usage académique)	Risque de conformité pour le "Objectif avancé"	Encadrant
4	Outil d'annotation retenu pour les 50-100 images Weizmann (CVAT, LabelMe, Label Studio, script maison)	Impacte le temps Semaine 7	Stagiaire
5	Dépôt Git : hébergement (GitHub Efrei, GitLab interne, GitHub public) et politique de visibilité	Impacte "Livrables attendus" (dépôt documenté)	Encadrant
6	Format et gabarit du rapport scientifique court et des 10-12 slides	Impacte Semaine 8	Encadrant
7	Niveau d'objectif visé dès le départ (minimal vs avancé) — le sujet propose les deux mais ne tranche pas	Redéfinit tout le backlog Sprint 3-4	Encadrant + stagiaire, à fixer en semaine 1
8	Disponibilité et format exact des checkpoints/scripts du dépôt <i>Learning from Synthetic Animals</i> au moment du stage (dépôt tiers, peut évoluer)	Semaine 3	Stagiaire, vérification technique

Hypothèse de travail retenue pour la suite de ce document (à valider en semaine 1) : on vise l'**objectif avancé**, avec repli possible sur l'objectif minimal si les données ou le calcul manquent — cf. section Risques.

1. Résumé exécutif

1.1 Objectifs du projet

- Reproduire une **version simplifiée mais fonctionnelle** du pipeline auto-supervisé du papier : image → squelette → pose 2D → lifting 3D → boucle de cohérence géométrique

+ discriminateur adversarial.

- Ne **pas** viser une reproduction parfaite des scores du papier (hors de portée en 2 mois), mais produire un pipeline **testable, documenté et évaluable**.
- Produire une évaluation qualitative systématique et, si le temps le permet, un score **PCK@0.05** quantitatif sur un sous-ensemble annoté de Weizmann Horse.

1.2 Périmètre (Scope)

Inclus :

- Collecte/préparation des données (TigDog cheval, extraction de frames YouTube, Weizmann Horse, prior synthétique CAD).
- Implémentation PyTorch des 6 composants du pipeline (image→squelette, squelette→pose 2D, lifting 2D→3D, rendu/pseudo-rendu, discriminateur, perte de cohérence géométrique).
- Entraînement sur sous-ensemble réduit, évaluation qualitative + quantitative partielle.
- Livrables : rapport, dépôt Git, notebook de visualisation, fiche datasets/licences, slides.

Exclu explicitement :

- Reproduction exacte des ~47k frames + 60 vidéos à pleine échelle (hors budget temps/calcul d'un stage de 2 mois).
- Développement d'un modèle 3D CAD complet du cheval (le papier lui-même l'évite).
- Mise en production, API, interface utilisateur, ou tout déploiement grand public — **ce n'est pas un projet logiciel commercial, c'est un projet de recherche reproductible**.
- Optimisation exhaustive des hyperparamètres (budget calcul limité).

1.3 Hypothèses

- H1 : Le stagiaire dispose de connaissances de base en PyTorch et vision par ordinateur (prérequis implicite du sujet).
- H2 : Un GPU (local ou distant) est accessible dès la semaine 1 — *à confirmer, cf. Required Inputs §1*.
- H3 : Les ressources publiques (TigDog, Weizmann Horse, dépôt Learning from Synthetic Animals) restent accessibles pendant toute la durée du stage.
- H4 : L'échelle de données sera réduite par rapport au papier original (sous-ensemble, pas totalité).

1.4 Contraintes

- Contrainte temporelle dure : 8 semaines, structure imposée par le planning du sujet.
- Absence de vérité terrain 3D pour les chevaux → évaluation limitée à la pose 2D (contrainte du domaine, pas du stage).

- Encadrement à distance (distanciel) → nécessité de points de synchronisation réguliers et de documentation continue plutôt que différée.

1.5 Critères de succès

Critère	Niveau minimal (obligatoire)	Niveau avancé (souhaité)
Pipeline fonctionnel end-to-end	Oui, sur sous-ensemble réduit	Oui, avec entraînement plus large
Visualisations qualitatives 2D/3D	Oui	Oui + comparaison visuelle avec baseline
Évaluation quantitative	Non requise	PCK@0.05 sur 50-100 images annotées
Baseline de comparaison	Non requise	Modèle 2D supervisé simple ou variante sans cohérence géométrique
Livrables documentaires	Rapport + dépôt + notebook + fiche licences	+ slides finalisées + comparaison chiffrée

1.6 Risques majeurs (résumé — détail en section 9)

1. Accès/qualité des données TigDog incertain.
2. Instabilité d'entraînement du GAN (discriminateur squelette) — risque classique et documenté sur ce type d'architecture.
3. Budget de calcul insuffisant pour l'objectif avancé.
4. Annotation manuelle Weizmann chronophage et sujette à erreur inter-annotateur (un seul stagiaire = pas de mesure d'accord inter-annotateur possible nativement).

2. Analyse fonctionnelle

2.1 Découpage des fonctionnalités (composants du pipeline)

ID	Composant	Fonction	Entrée	Sortie
F1	Image → Squelette	Extraire une représentation squelette/heatmap à partir de l'image RGB	Image 128×128	Carte de squelette / heatmaps
F2	Squelette → Pose 2D	Régresser les coordonnées des points clés 2D	Heatmaps	Coordonnées 2D des keypoints

ID	Composant	Fonction	Entrée	Sortie
F3	Lifting 2D → 3D	Estimer une pose 3D plausible à partir des points 2D	Points 2D	Pose 3D (angles/joints)
F4	Rendu / pseudo-rendu	Reprojeter la pose 3D en squelette 2D pour la cohérence	Pose 3D	Squelette 2D reprojété
F5	Discriminateur adversarial	Juger la plausibilité d'un squelette (réel synthétique vs prédit)	Squelette 2D	Score réaliste/non réaliste
F6	Cohérence géométrique	Vérifier la cohérence sous rotations/projections 3D aléatoires	Pose 3D	Perte de cohérence

2.2 User stories (formulées du point de vue du stagiaire-chercheur / de l'encadrant)

ID	User story	Priorité
US1	En tant que stagiaire, je veux télécharger et prétraiter TigDog + Weizmann pour disposer d'un jeu de données exploitable	Must
US2	En tant que stagiaire, je veux extraire des frames de vidéos YouTube pour enrichir les données non annotées	Should
US3	En tant que stagiaire, je veux récupérer le prior synthétique 2D (10k poses CAD) pour entraîner le discriminateur	Must
US4	En tant que stagiaire, je veux implémenter chaque composant (F1-F6) de façon isolée et testable	Must
US5	En tant que stagiaire, je veux assembler le pipeline complet et lancer un entraînement sur sous-ensemble réduit	Must
US6	En tant que stagiaire, je veux visualiser qualitativement les poses 2D/3D prédites	Must
US7	En tant qu'encadrant, je veux un score PCK@0.05 quantitatif pour évaluer objectivement le résultat	Should
US8	En tant qu'encadrant, je veux un baseline de comparaison pour contextualiser les résultats	Should
US9	En tant que stagiaire, je veux un dépôt Git documenté pour que le travail soit reproductible après le stage	Must
US10	En tant qu'encadrant, je veux un rapport et une présentation synthétiques pour évaluer le stage	Must

- **UC1 — Préparer les données** : télécharger TigDog/Weizmann/vidéos, filtrer les frames avec cheval visible, redimensionner en 128×128, générer les masques.
- **UC2 — Préparer le prior synthétique** : extraire/générer 10k poses 2D CAD, normaliser, structurer en dataset PyTorch.
- **UC3 — Entraîner le modèle** : boucle d'entraînement combinant perte adversariale + perte de cohérence géométrique.
- **UC4 — Évaluer** : inférence sur Weizmann Horse, visualisation, calcul PCK@0.05 sur sous-ensemble annoté.
- **UC5 — Documenter et livrer** : rapport, dépôt, notebook, fiche licences, slides.

2.4 Règles métier (règles méthodologiques du papier à respecter)

- **RM1** : Aucune annotation 2D/3D des images d'entraînement réelles n'est utilisée (le principe auto-supervisé doit être respecté, sinon la reproduction perd son sens scientifique).
- **RM2** : Le prior synthétique reste la seule source de supervision indirecte sur la plausibilité des poses.
- **RM3** : L'évaluation quantitative se fait uniquement en 2D (pas de vérité terrain 3D disponible — contrainte inhérente au domaine).
- **RM4** : Les images doivent être redimensionnées en 128×128 comme dans le papier, pour rester comparable méthodologiquement.

2.5 Cas limites (edge cases)

Cas limite	Composant concerné	Traitement prévu
Frame vidéo sans cheval visible ou cheval partiellement hors-cadre	Prétraitement données	Filtrage automatique (détecteur simple ou seuil de confiance) + vérification manuelle sur échantillon
Masque de segmentation de mauvaise qualité (bruité)	F1	Seuil de qualité minimal ; rejet si IoU estimé trop faible
Divergence du discriminateur (mode collapse classique en GAN)	F5	Monitoring de la loss adversariale ; arrêt anticipé (early stopping) si divergence détectée
Pose 3D non physiquement plausible après lifting	F3/F6	Contraintes de cohérence géométrique ; clipping des angles articulaires
Image Weizmann hors distribution (angle de vue	Évaluation	Exclusion documentée du sous-ensemble annoté, avec justification dans le rapport

Cas limite	Composant concerné	Traitement prévu
extrême)		

2.6 Scénarios d'erreur

- Échec de téléchargement (lien TigDog/vidéos indisponible) → script de retry + log + notification manuelle.
- Incompatibilité de version PyTorch/CUDA avec le dépôt *Learning from Synthetic Animals* → environnement isolé (voir §4.6) avec pinning strict des versions.
- Entraînement qui diverge (loss → NaN) → checkpointing fréquent + reprise automatique depuis le dernier checkpoint stable.
- Annotation manuelle incohérente → double passage sur un petit échantillon pilote pour calibrer la procédure d'annotation avant les 50-100 images définitives.

3. Architecture technique

3.1 Architecture haut niveau

```
flowchart LR
    subgraph Données
        A1[TigDog - cheval]
        A2[Frames YouTube]
        A3[Weizmann Horse]
        A4[Prior synthétique CAD 2D]
    end

    subgraph Prétraitement
        B1[Filtrage + resize 128x128]
        B2[Masques de segmentation]
        B3[Normalisation poses synthétiques]
    end

    subgraph Pipeline modèle
        C1[F1: Image -> Squelette]
        C2[F2: Squelette -> Pose 2D]
        C3[F3: Lifting 2D -> 3D]
        C4[F4: Rendu / pseudo-rendu]
        C5[F5: Discriminateur]
        C6[F6: Cohérence géométrique]
    end

    subgraph Évaluation
        D1[Visualisation qualitative]
        D2[PCK@0.05]
        D3[Comparaison baseline]
    end
```

end

A1 --> B1

A2 --> B1

B1 --> B2

A3 --> D1

A4 --> B3

B2 --> C1 --> C2 --> C3 --> C4

C4 --> C5

B3 --> C5

C3 --> C6

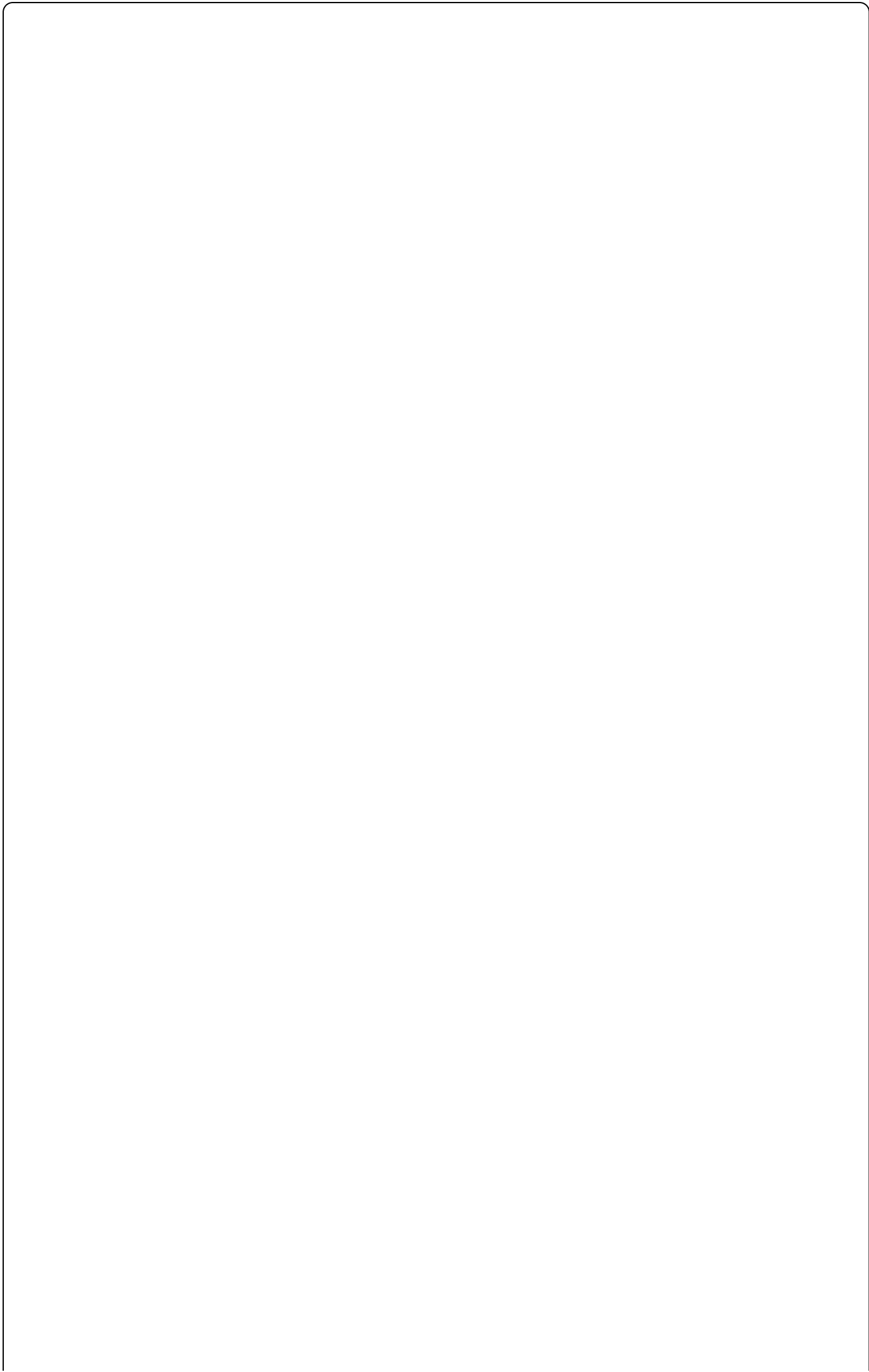
C2 --> D1

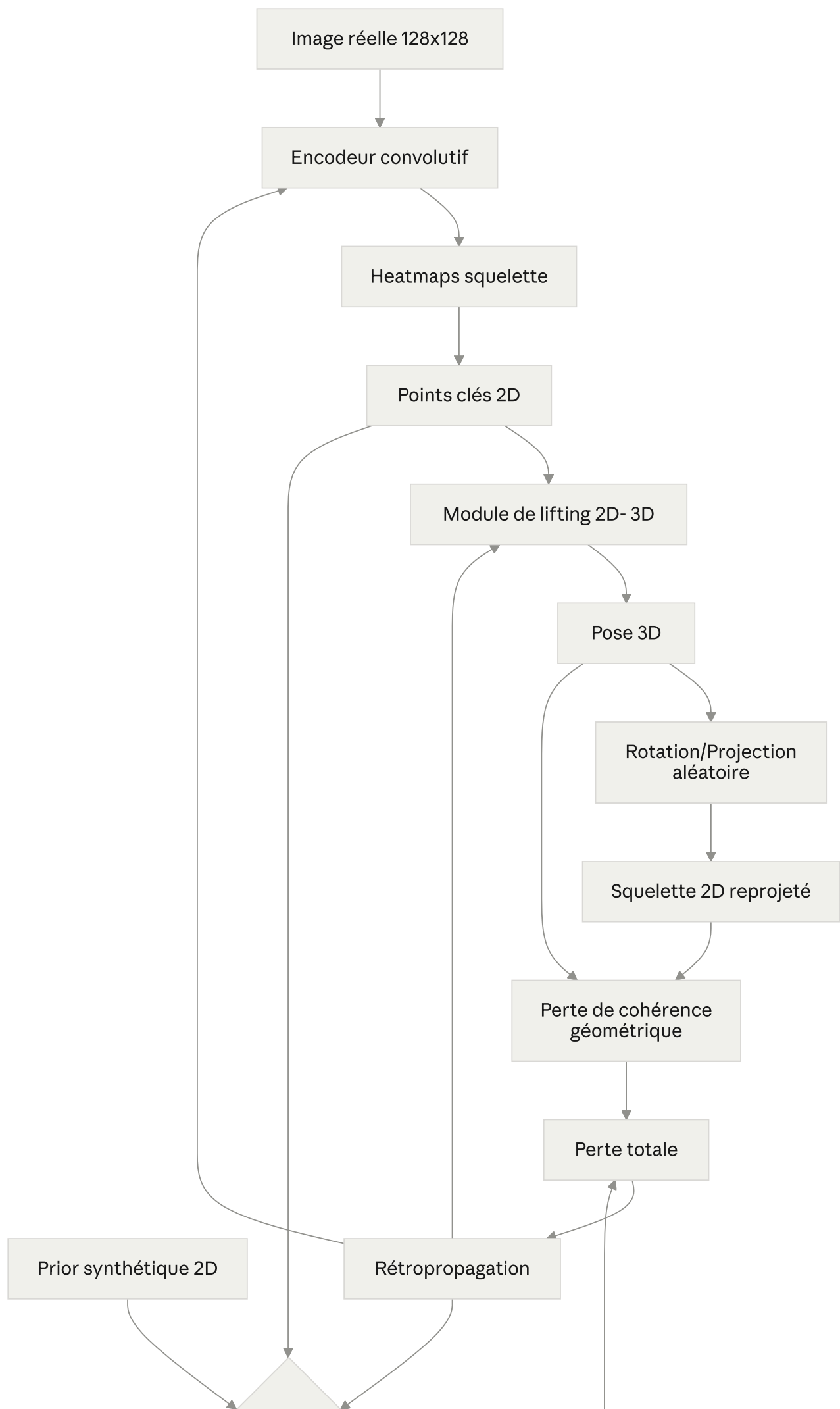
C3 --> D1

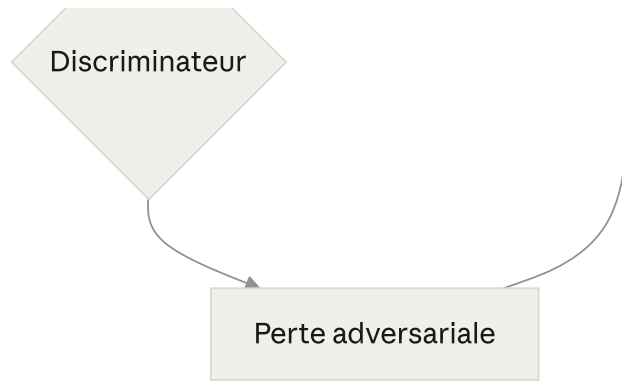
D1 --> D2

D2 --> D3

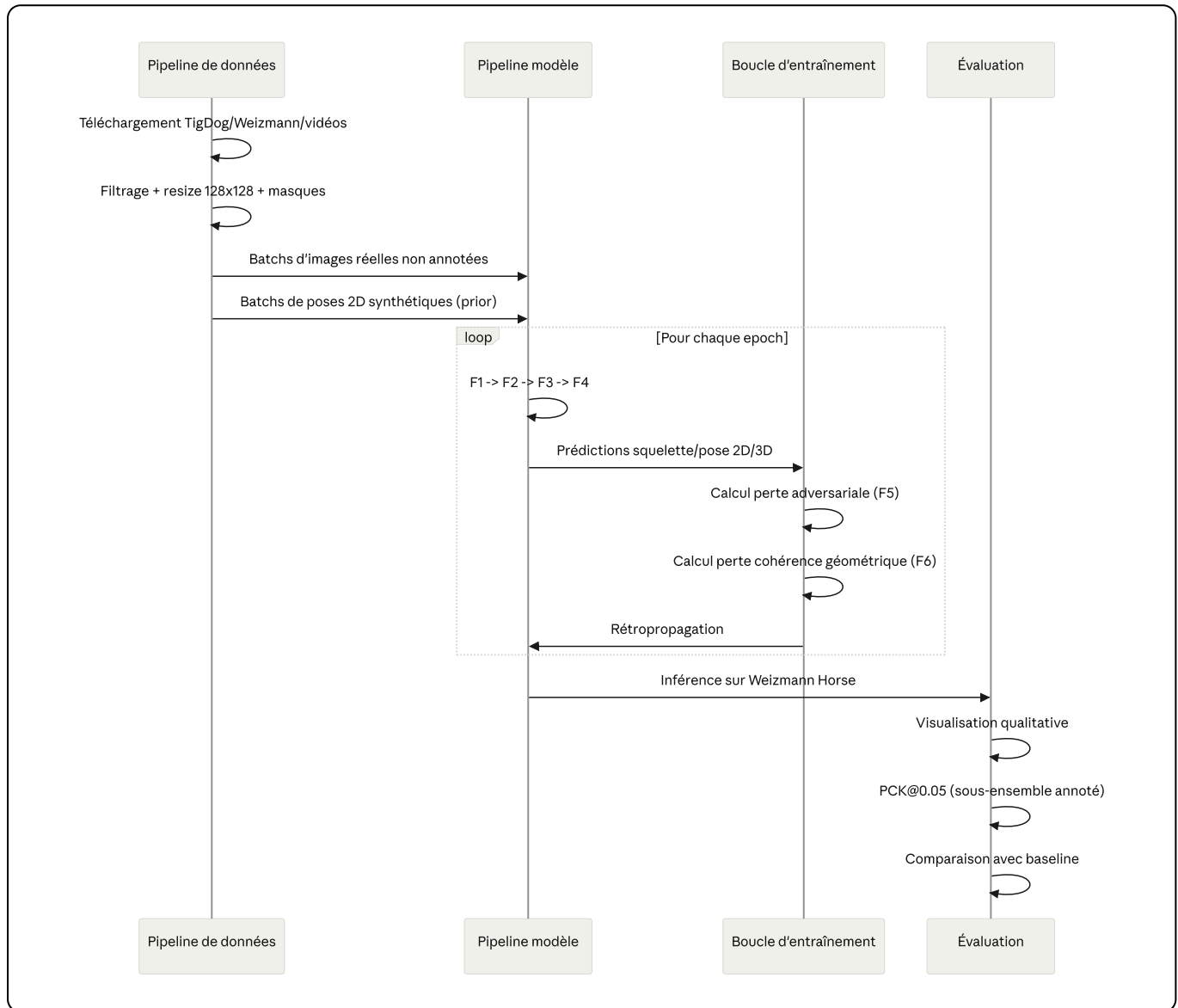
3.2 Architecture des composants (détail du pipeline modèle)







3.3 Flux de données (data flow)



3.4 Architecture de déploiement (contexte recherche, pas production)

Ce projet n'a **pas** vocation à être déployé en production. L'« architecture de déploiement » se limite à un environnement d'exécution reproductible pour la recherche :

Élément	Choix retenu	Justification
Environnement d'exécution	Conteneur Docker ou environnement conda figé	Reproductibilité, portabilité entre poste local et éventuel cluster
Gestion de versions de données	DVC (Data Version Control) ou simple registre de checksums dans le dépôt	Traçabilité des sous-ensembles utilisés à chaque entraînement
Suivi d'expériences	Weights & Biases (mode gratuit) ou TensorBoard local	Suivi des courbes de loss, détection de divergence GAN
Stockage des checkpoints	Dossier <code>checkpoints/</code> versionné hors Git (via <code>.gitignore</code>) + lien externe)	Les poids de modèle ne doivent pas être commités dans Git

3.5 Exigences d'infrastructure

- GPU avec au moins 8 Go de VRAM recommandé pour un entraînement en résolution 128×128 avec batchs raisonnables (à confirmer selon Required Inputs §1).
- Espace disque : prévoir large marge pour les frames vidéo brutes avant filtrage (~plusieurs dizaines de Go potentiellement, à réévaluer après Semaine 2).
- Connexion réseau stable pour le téléchargement initial des datasets et vidéos (Semaine 1).

3.6 Justification de la pile technologique

Choix	Alternative envisagée	Justification
PyTorch	TensorFlow	Imposé par le sujet ; écosystème compatible avec le dépôt <i>Learning from Synthetic Animals</i>
OpenCV / FFmpeg	Autres outils d'extraction vidéo	Standards, robustes, bien documentés pour l'extraction de frames YouTube
yt-dlp	youtube-dl	Maintenu plus activement, meilleure compatibilité actuelle
Weights & Biases / TensorBoard	MLflow	Plus léger à mettre en place pour un projet de 2 mois en solo
DVC (optionnel)	Gestion manuelle de fichiers	Traçabilité sans complexité excessive

4. Architecture "backend" (organisation du code du pipeline)

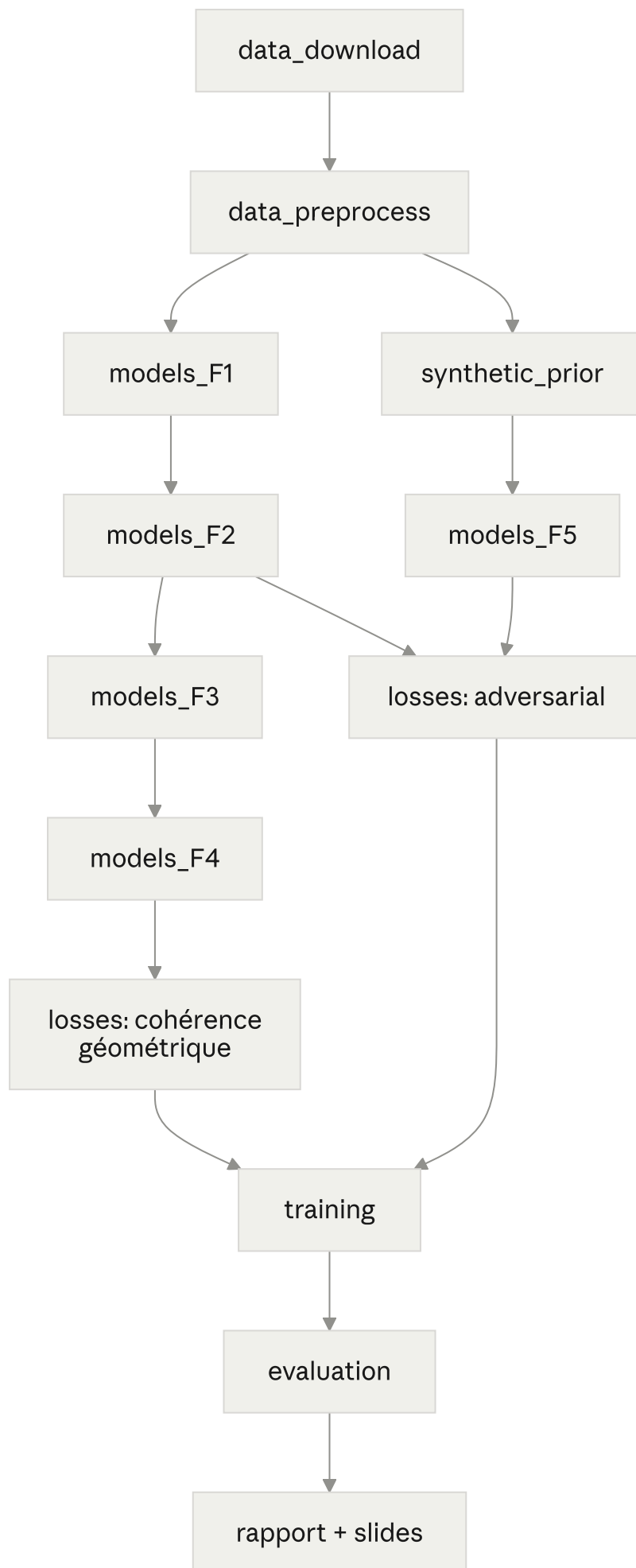
Note : le terme "backend" est repris du gabarit générique du sujet de stage. Dans ce contexte de recherche, il désigne l'organisation modulaire du code Python/PyTorch, et non un service applicatif.

4.1 Modules du dépôt

```
horse-pose-repro/
├─ data/
│   ├── download_tigdog.py
│   ├── download_weizmann.py
│   ├── extract_youtube_frames.py
│   ├── preprocess.py          # filtrage, resize 128x128, masques
│   └─ synthetic_prior.py      # préparation du prior CAD 2D
├─ models/
│   ├── image_to_skeleton.py   # F1
│   ├── skeleton_to_pose2d.py  # F2
│   ├── lifting_2d_3d.py       # F3
│   ├── renderer.py           # F4
│   └─ discriminator.py        # F5
├─ losses/
│   ├── adversarial_loss.py
│   └─ geometric_consistency.py # F6
├─ training/
│   ├── train.py
│   ├── config.yaml
│   └─ checkpointing.py
├─ evaluation/
│   ├── inference.py
│   ├── visualize.py
│   ├── pck.py
│   └─ baseline_comparison.py
├─ notebooks/
│   └─ visualization_demo.ipynb
├─ tests/
│   ├── test_data_pipeline.py
│   ├── test_models.py
│   └─ test_losses.py
├─ docs/
│   ├── report.md
│   ├── dataset_license_sheet.md
│   └─ runbook.md
├─ requirements.txt / environment.yml
└─ README.md
```

4.2 Logique métier par module

- **data/** : responsable exclusivement de l'acquisition et du nettoyage. Aucune logique de modèle ici.
- **models/** : chaque composant F1-F5 est un `nn.Module` indépendant, testable isolément (entrée/sortie mockées).
- **losses/** : séparées des modèles pour permettre de tester des combinaisons de pertes sans réentraîner.
- **training/** : orchestration uniquement ; ne redéfinit pas la logique métier des modules.
- **evaluation/** : totalement découplée de l'entraînement, consomme uniquement des checkpoints sauvegardés.



4.4 Tâches de fond (background jobs)

Dans ce contexte, les "jobs" sont des scripts d'entraînement longue durée plutôt que des tâches applicatives :

- Extraction de frames vidéo (potentiellement plusieurs heures) → à lancer en amont, en tâche isolée, avec logs de progression.
- Entraînement principal → à lancer via script avec reprise sur checkpoint (`--resume`) en cas d'interruption.
- Génération périodique de visualisations pendant l'entraînement (toutes les N epochs) pour un contrôle qualité continu.

4.5 Cache

- Mise en cache locale des frames prétraitées (éviter de refaire l'extraction/redimensionnement à chaque run).
- Cache des features/heatmaps intermédiaires uniquement si le profilage montre un goulot d'étranglement (à ne pas sur-ingénierer dans un contexte de stage).

4.6 Stratégie de scalabilité (adaptée à un stage, pas à un produit)

- Démarrer sur un **sous-ensemble volontairement réduit** (ex. 2-5k frames) pour valider le pipeline end-to-end rapidement (fail-fast), puis augmenter progressivement l'échelle en semaine 6-7 si le calcul le permet.
- Paramétrer la taille du dataset via `config.yaml` pour ne jamais coder en dur une taille d'échantillon.

5. Sécurité (adaptée au contexte recherche)

Le projet ne traite pas de données personnelles sensibles au sens réglementaire classique, mais des exigences existent néanmoins.

5.1 Modèle de menace simplifié

Menace	Probabilité	Impact	Mitigation
Usage de vidéos YouTube sans droits clairs	Moyenne	Conformité / retrait possible du dépôt	Documenter la provenance, usage strictement académique, ne pas republier les vidéos brutes
Fuite de checkpoints volumineux dans Git	Élevée si non anticipé	Dépôt inutilisable / historique pollué	<code>.gitignore</code> strict sur <code>checkpoints/</code> , <code>data/raw/</code>

Menace	Probabilité	Impact	Mitigation
Dépendance à un dépôt tiers non maintenu (<i>Learning from Synthetic Animals</i>)	Moyenne	Blocage Semaine 3	Fork/mirror local dès la première utilisation
Exécution de code tiers non audité (dépôt GitHub externe)	Faible-Moyenne	Risque d'exécution de code non désiré	Revue rapide du code avant exécution, environnement conteneurisé isolé

5.2 Authentication / Autorisation

Non applicable au sens applicatif — pas de système multi-utilisateur. Si un espace de stockage partagé (cluster Efrei) est utilisé : respecter les accès et quotas définis par l'organisme d'accueil (cf. Required Inputs §1).

5.3 Gestion des secrets

- Aucun secret métier attendu. Si une clé API est utilisée (ex. Weights & Biases), la stocker en variable d'environnement, jamais en dur dans le code, et l'exclure du dépôt via `.env` + `.gitignore`.

5.4 Conformité et licences

- Obligatoire** : fiche de vérification des datasets et de leurs licences (déjà listée dans les livrables attendus du sujet). Doit couvrir : TigDog, Weizmann Horse, prior synthétique CAD, vidéos YouTube extraites.
- Vérifier explicitement si TigDog/Weizmann autorisent un usage académique de reproduction et de republication de résultats dérivés (pas seulement de consultation).

6. Stratégie de validation (adaptée à chaque étape du pipeline)

Couche	Validation prévue
Validation des données d'entrée	Vérifier dimensions (128×128), présence de masque, absence de frame corrompue, cheval effectivement visible (échantillon contrôlé manuellement)
Validation des règles métier (RM1-RM4)	Revue de code + revue du rapport pour confirmer qu'aucune annotation réelle n'a fuité dans l'entraînement
Validation de schéma des configs	<code>config.yaml</code> validé via schéma simple (types, valeurs autorisées) avant lancement d'un entraînement

Couche	Validation prévue
Validation des modules modèle	Tests unitaires sur formes de tenseurs en entrée/sortie de chaque module F1-F6
Validation d'entraînement	Vérification que la loss totale ne diverge pas (NaN/Inf) sur un mini-batch avant lancement complet
Validation de reprise sur checkpoint	Test explicite : interrompre un entraînement, reprendre, vérifier continuité de la loss
Validation de l'évaluation	Vérifier manuellement un sous-échantillon des points clés PCK sur quelques images avant calcul global
Validation de configuration d'environnement	<code>environment.yml</code> / <code>requirements.txt</code> testé sur environnement propre avant Semaine 4
Validation de la documentation	Relecture croisée (encadrant) du rapport et de la fiche licences avant dépôt final

(Les catégories "accessibilité", "localisation", "API", "client-side/server-side" du gabarit générique ne sont pas pertinentes pour ce projet de recherche sans interface utilisateur ; elles sont volontairement omises plutôt que remplies artificiellement.)

7. Stratégie de test

Type de test	Application au projet	Quand
Tests unitaires	Formes de tenseurs, comportement de chaque module F1-F6 isolément	Semaine 4-5, en continu
Tests d'intégration	Pipeline complet sur un micro-batch (2-4 images) de bout en bout	Fin Semaine 5
Tests de non-régression	Rejouer un mini-entraînement de référence après chaque modification majeure de perte	Semaine 5-7
Smoke test	Lancer 1 epoch sur données factices avant tout run long	Avant chaque run réel
Test de charge / stress	Non prioritaire (pas de système multi-utilisateur) ; à minima, tester avec la taille de batch maximale prévue	Semaine 6
Test de robustesse aux données limites	Cas limites du §2.5 (masques bruités, poses non plausibles)	Semaine 6

Type de test	Application au projet	Quand
QA manuelle	Revue visuelle systématique des poses prédites (encadrant + stagiaire)	Chaque fin de sprint
Couverture de test	Cible réaliste pour un projet de recherche solo : couverture fonctionnelle des modules critiques (F1-F6, pertes), pas un pourcentage LOC formel	Continu
Critères de sortie (exit criteria)	Pipeline exécutable end-to-end sans erreur sur sous-ensemble réduit + visualisations cohérentes	Fin Semaine 6

8. Feuille de route agile incrémentale (alignée sur le planning 8 semaines du sujet)

Sprint 1 (Semaine 1) — Cadrage et accès aux données

- **Objectifs** : lecture du papier, installation environnement, accès TigDog/Weizmann/synthétique.
- **Livrables** : environnement reproductible (Docker/conda), synthèse bibliographique courte, accès aux 3 sources de données confirmé.
- **User stories couvertes** : US1 (partiel), US3 (partiel).
- **Étapes détaillées** :
 1. **Jour 1 — Cadrage administratif et bibliographique** 1.1. Réunion de lancement avec l'encadrant : trancher explicitement Required Inputs §1 (calcul), §7 (niveau d'objectif minimal vs avancé) et §5 (hébergement Git). 1.2. Lecture complète du papier "*A Horse with no Labels*" (ICCVW 2023) ; prise de notes structurées par composant (F1-F6). 1.3. Identifier et lister les 2-3 travaux directement cités comme base (self-supervised 3D pose estimation, données synthétiques animales).
 2. **Jour 2 — Étude bibliographique ciblée** 2.1. Lire en détail le travail de self-supervised 3D pose estimation dont s'inspire le papier ; noter les différences d'implémentation attendues. 2.2. Lire la documentation du dépôt *Learning from Synthetic Animals* (README, structure du code, scripts disponibles). 2.3. Rédiger une synthèse bibliographique courte (1-2 pages) : problème, approche, composants, limites connues.
 3. **Jour 3 — Mise en place de l'environnement** 3.1. Créer l'environnement (conda ou Docker) avec Python + PyTorch + CUDA compatible avec le GPU alloué. 3.2. Installer les dépendances de base : `torch`, `torchvision`, `opencv-python`, `numpy`, `matplotlib`, `pyyaml`. 3.3. Vérifier l'accès GPU (`torch.cuda.is_available()`), documenter la configuration matérielle dans le README. 3.4. Initialiser le dépôt Git (structure de dossiers du §4.1), premier commit.

4. **Jour 4 — Accès aux données** 4.1. Identifier et tester le lien de téléchargement TigDog (sous-ensemble cheval) ; documenter la procédure exacte. 4.2. Identifier et tester le lien de téléchargement Weizmann Horse. 4.3. Écrire un script `data/download_tigdog.py` et `data/download_weizmann.py` (téléchargement + vérification d'intégrité par checksum). 4.4. En cas de blocage d'accès : escalader immédiatement à l'encadrant (ne pas attendre — cf. Risque R1).
 5. **Jour 5 — Premier contact avec le dépôt tiers et synthèse** 5.1. Cloner/forker *Learning from Synthetic Animals* dans un sous-module ou miroir versionné. 5.2. Tester l'installation de ses dépendances dans l'environnement du stage (repérer les incompatibilités de version). 5.3. Vérifier concrètement la disponibilité du dataset synthétique CAD, des scripts de prétraitement TigDog et des checkpoints mentionnés. 5.4. Point de synchronisation de fin de semaine avec l'encadrant : validation environnement + accès données + confirmation du niveau d'objectif retenu.
- **Dépendances** : Required Inputs §1, §2, §8 résolus.
 - **Critères d'acceptation** : environnement fonctionnel testé (`import torch;`
`torch.cuda.is_available()`) documenté ; au moins un dataset téléchargé avec succès.
 - **Points de validation** : revue avec encadrant en fin de semaine.
 - **Tests requis** : test manuel d'installation sur poste propre.
 - **Risques** : accès données bloqué (cf. Risque R1).
 - **Definition of Done** : environnement + accès données validés par l'encadrant.
 - **Effort estimé** : 5 jours.

Sprint 2 (Semaine 2) — Prétraitement des données réelles

- **Objectifs** : extraction de frames, filtrage cheval visible, resize 128×128, masques.
- **Livrables** : `data/preprocess.py` fonctionnel, dataset prétraité versionné (checksums).
- **User stories** : US1, US2.
- **Étapes détaillées** :
 1. **Jour 1 — Finalisation des données TigDog/Weizmann** 1.1. Terminer le téléchargement complet de TigDog (sous-ensemble cheval) et Weizmann Horse. 1.2. Vérifier l'intégrité (checksums) et la structure des fichiers reçus (formats d'image, résolutions d'origine). 1.3. Recenser précisément le volume disponible (nombre de frames, nombre de séquences).
 2. **Jour 2 — Extraction des frames vidéo YouTube** 2.1. Sélectionner la liste des ~60 vidéos ciblées (ou sous-ensemble réduit selon Required Inputs §1/§7). 2.2. Écrire `data/extract_youtube_frames.py` avec `yt-dlp` (téléchargement) + `ffmpeg` (extraction à intervalle régulier). 2.3. Lancer l'extraction sur 3-5 vidéos pilotes pour valider le script avant la volumétrie complète. 2.4. Documenter dans la fiche licences (§5.4) la provenance de chaque vidéo (lien, chaîne, date d'extraction).
 3. **Jour 3 — Filtrage automatique** 3.1. Implémenter un filtre de présence du cheval (détecteur simple pré-entraîné ou heuristique de mouvement/contraste, à

trancher avec l'encadrant). 3.2. Appliquer le filtre sur l'ensemble des frames extraites ; produire un rapport de taux de rejet. 3.3. Contrôle manuel sur un échantillon aléatoire (~100 frames) pour valider la qualité du filtre.

4. **Jour 4 — Redimensionnement et masques** 4.1. Implémenter le redimensionnement 128×128 dans `data/preprocess.py` (avec gestion du ratio d'aspect, padding si nécessaire). 4.2. Générer les masques de segmentation automatiques (outil à définir : modèle de segmentation pré-entraîné générique, ou scripts fournis par *Learning from Synthetic Animals* si disponibles pour TigDog). 4.3. Vérifier visuellement un échantillon de masques (superposition masque/image).
5. **Jour 5 — Consolidation et tests** 5.1. Assembler le dataset prétraité final (images 128×128 + masques) dans une structure de dossiers versionnée par checksums. 5.2. Écrire `tests/test_data_pipeline.py` : vérification des dimensions, absence de corruption, cohérence image/masque. 5.3. Rédiger le rapport de qualité de prétraitement (taux de rejet, volumétrie finale) et le partager à l'encadrant.

- **Dépendances** : Sprint 1 terminé.
- **Critères d'acceptation** : X% (à définir en semaine 1) des frames extraites passent le filtre qualité ; masques générés visuellement cohérents sur échantillon de contrôle.
- **Tests requis** : `tests/test_data_pipeline.py` (dimensions, absence de corruption).
- **Risques** : R2 (temps d'extraction vidéo sous-estimé).
- **Definition of Done** : dataset prétraité + rapport de qualité (taux de rejet documenté).
- **Effort estimé** : 5 jours.

Sprint 3 (Semaine 3) — Prior synthétique 2D

- **Objectifs** : extraction/normalisation des ~10k poses CAD.
- **Livrables** : `data/synthetic_prior.py`, dataset synthétique prêt pour le discriminateur.
- **User stories** : US3.
- **Étapes détaillées** :
 1. **Jour 1 — Intégration du dépôt tiers** 1.1. Explorer en détail la structure du dépôt *Learning from Synthetic Animals* (scripts de génération, dataset synthétique, checkpoints). 1.2. Identifier précisément le sous-ensemble "cheval" du dataset synthétique CAD (~10k poses attendues). 1.3. Adapter/exécuter les scripts de génération pour extraire les poses CAD spécifiques au cheval.
 2. **Jour 2 — Extraction des points clés synthétiques** 2.1. Écrire `data/synthetic_prior.py` : chargement des poses CAD brutes et extraction des coordonnées des points clés 2D. 2.2. Définir explicitement le schéma de points clés retenu (nombre et nom des joints — ex. tête, garrot, hanches, sabots) et le documenter. 2.3. Vérifier la cohérence du schéma de points clés avec celui utilisé pour l'évaluation Weizmann (alignement obligatoire pour PCK@0.05 en Sprint 7).

3. **Jour 3 — Rendu des squelettes synthétiques** 3.1. Implémenter une fonction de rendu simple (squelette = ensemble de segments reliant les points clés) pour visualisation et pour servir d'entrée au discriminateur (F5). 3.2. Générer et sauvegarder des visualisations pour un échantillon de 20 poses synthétiques.
 4. **Jour 4 — Normalisation des poses** 4.1. Implémenter la normalisation (échelle, centrage, éventuellement rotation canonique) pour homogénéiser les poses synthétiques. 4.2. Vérifier statistiquement la distribution des poses normalisées (histogrammes de coordonnées) pour détecter des anomalies.
 5. **Jour 5 — Intégration DataLoader et validation** 5.1. Encapsuler le prior synthétique dans un `(torch.utils.data.Dataset) + (DataLoader)` PyTorch. 5.2. Tester le chargement par batch (vérifier formes de tenseurs, absence de NaN). 5.3. Étendre `(tests/test_data_pipeline.py)` au prior synthétique. 5.4. Point de synchronisation : validation du prior synthétique par l'encadrant avant de démarrer l'implémentation du discriminateur (Sprint 5).
- **Dépendances** : accès confirmé au dépôt tiers (Required Inputs §8).
 - **Critères d'acceptation** : dataset synthétique chargeable en `(DataLoader)` PyTorch, vérifié visuellement sur 10 échantillons.
 - **Tests requis** : `(test_data_pipeline.py)` étendu au prior synthétique.
 - **Risques** : R3 (dépôt tiers non maintenu / incompatible).
 - **Definition of Done** : prior synthétique intégré et testé.
 - **Effort estimé** : 5 jours.

Sprint 4 (Semaine 4) — Implémentation du pipeline modèle (F1-F4)

- **Objectifs** : implémenter image→squelette, squelette→pose 2D, lifting 2D→3D, rendu/pseudo-rendu.
- **Livrables** : modules `(models/)` fonctionnels et testés isolément.
- **User stories** : US4.
- **Étapes détaillées** :
 1. **Jour 1 — Module F1 (Image → Squelette)** 1.1. Concevoir l'encodeur convolutif (ex. architecture type hourglass/U-Net légère adaptée à des images 128×128). 1.2. Implémenter `(models/image_to_skeleton.py)` : sortie sous forme de heatmaps (une par point clé). 1.3. Test unitaire : forme de sortie correcte sur un batch factice.
 2. **Jour 2 — Module F2 (Squelette → Pose 2D)** 2.1. Implémenter `(models/skeleton_to_pose2d.py)` : conversion heatmaps → coordonnées 2D (ex. soft-argmax pour rester différentiable). 2.2. Vérifier que la sortie respecte le schéma de points clés défini en Sprint 3 (même nombre/ordre de joints). 2.3. Test unitaire de cohérence des formes F1→F2.
 3. **Jour 3 — Module F3 (Lifting 2D → 3D)** 3.1. Documenter explicitement l'interprétation retenue pour ce module (sous-spécifié dans le papier — cf. Risque R4), avec justification dans le rapport. 3.2. Implémenter

`models/lifting_2d_3d.py` (ex. MLP par joint ou réseau partagé, prédisant profondeur/angles). 3.3. Ajouter des contraintes simples de plausibilité (bornes d'angles articulaires).

4. Jour 4 — Module F4 (Rendu / pseudo-rendu) 4.1. Implémenter

`models/renderer.py` : reprojection de la pose 3D en squelette 2D via une projection paramétrable (caméra simplifiée). 4.2. Implémenter la rotation/projection aléatoire nécessaire à la cohérence géométrique (réutilisée en Sprint 5 pour F6). 4.3. Test unitaire : reprojection d'une pose 3D connue redonne un squelette 2D cohérent (aux erreurs numériques près).

5. Jour 5 — Assemblage forward-only et tests 5.1. Assembler F1→F2→F3→F4 dans un unique forward pass sans entraînement. 5.2. Exécuter ce forward pass sur un batch réel (données Sprint 2) et un batch synthétique (données Sprint 3). 5.3. Écrire `tests/test_models.py` couvrant chaque module isolément + l'assemblage complet.

6. Jour 6 (marge) — Revue de code et stabilisation 6.1. Revue de code (auto-revue ou avec l'encadrant) sur l'ensemble F1-F4. 6.2. Corriger les incohérences de forme de tenseurs ou de device (CPU/GPU) détectées. 6.3. Documenter les choix d'architecture dans `docs/report.md` (section méthode, brouillon).

- **Dépendances** : Sprints 2 et 3 terminés (données disponibles).
- **Critères d'acceptation** : forward pass complet sans erreur sur un batch réel + un batch synthétique.
- **Tests requis** : `tests/test_models.py`.
- **Risques** : R4 (choix d'architecture du module de lifting sous-spécifié dans le papier — nécessite interprétation documentée).
- **Definition of Done** : pipeline forward complet, revue de code effectuée.
- **Effort estimé** : 6 jours (légèrement au-delà de la semaine stricte, à absorber en marge Sprint 5).

Sprint 5 (Semaine 5) — Pertes et discriminateur (F5-F6)

- **Objectifs** : perte adversariale + perte de cohérence géométrique.
- **Livrables** : `losses/`, `models/discriminator.py`, script d'entraînement `training/train.py` fonctionnel.
- **User stories** : US4, US5 (partiel).
- **Étapes détaillées** :
 1. **Jour 1 — Module F5 (Discriminateur)** 1.1. Concevoir un discriminateur simple opérant sur les squelettes 2D (ex. petit MLP ou CNN léger sur les coordonnées/heatmaps de squelette). 1.2. Implémenter `models/discriminator.py` ; entrée = squelette réel (synthétique) OU prédit, sortie = score de plausibilité. 1.3. Test unitaire de forme sur squelette synthétique et squelette prédit.

2. **Jour 2 — Perte adversariale** 2.1. Implémenter `losses/adversarial_loss.py` (formulation GAN standard ou variante moins-square, à documenter et justifier). 2.2. Câbler l'alternance d'entraînement générateur/discriminateur dans une boucle de test minimale. 2.3. Vérifier numériquement que la loss reste finie sur quelques itérations.
3. **Jour 3 — Perte de cohérence géométrique** 3.1. Implémenter `losses/geometric_consistency.py` en réutilisant la rotation/projection aléatoire de F4 (Sprint 4). 3.2. Définir la métrique de cohérence (ex. distance L2 entre squelette reprojeté après rotation et squelette original reprojeté). 3.3. Test unitaire : la perte est nulle (ou quasi) quand aucune rotation n'est appliquée (cas de contrôle).
4. **Jour 4 — Assemblage de la boucle d'entraînement** 4.1. Écrire `training/train.py` : boucle combinant forward F1-F4, calcul perte adversariale (F5) + perte de cohérence (F6), rétropropagation. 4.2. Écrire `training/config.yaml` (hyperparamètres, chemins de données, pondération des pertes). 4.3. Implémenter `training/checkpointing.py` (sauvegarde périodique + reprise `--resume`).
5. **Jour 5 — Test d'intégration micro-batch** 5.1. Lancer une epoch complète sur un micro-batch (2-4 images réelles + 2-4 poses synthétiques). 5.2. Vérifier l'absence de NaN/Inf sur la loss totale et sur les gradients. 5.3. Écrire `tests/test_losses.py` + test d'intégration bout-en-bout automatisé. 5.4. Point de synchronisation : validation du pipeline entraînable avec l'encadrant avant de lancer les entraînements réels (Sprint 6).

- **Dépendances** : Sprint 4 terminé.
- **Critères d'acceptation** : une epoch complète sur micro-batch s'exécute sans erreur, loss totale finie (pas de NaN).
- **Tests requis** : `tests/test_losses.py` + test d'intégration bout-en-bout.
- **Risques** : R5 (instabilité GAN).
- **Definition of Done** : pipeline entraînable de bout en bout sur données factices.
- **Effort estimé** : 5 jours.

Sprint 6 (Semaine 6) — Premiers entraînements réels

- **Objectifs** : entraînement sur sous-ensemble réduit, validation qualitative, correction d'instabilités.
- **Livrables** : premiers checkpoints, courbes de loss (W&B/TensorBoard), visualisations qualitatives.
- **User stories** : US5, US6.
- **Étapes détaillées** :
 1. **Jour 1 — Mise en place du monitoring** 1.1. Configurer Weights & Biases ou TensorBoard (courbes de loss adversariale, loss de cohérence, loss totale). 1.2. Ajouter la génération périodique de visualisations (toutes les N epochs) au script d'entraînement.

2. **Jour 2 — Premier entraînement court** 2.1. Lancer un entraînement sur un sous-ensemble volontairement réduit (2-5k frames, cf. §4.6) pour valider le pipeline en conditions réelles (fail-fast). 2.2. Surveiller la stabilité numérique dès les premières itérations (loss finie, pas d'explosion de gradient).
 3. **Jour 3 — Détection et correction d'instabilités** 3.1. Analyser les courbes de loss : détecter mode collapse du discriminateur ou divergence. 3.2. Si instabilité détectée : appliquer les mitigations prévues au Risque R5 (learning rate réduit, pondération adversariale réduite, pré-entraînement F1-F4 seul avant activation du GAN). 3.3. Relancer un entraînement court avec les ajustements.
 4. **Jour 4 — Test de reprise sur checkpoint et passage à l'échelle** 4.1. Interrompre volontairement un entraînement, vérifier la reprise via `--resume` et la continuité de la courbe de loss. 4.2. Décider avec l'encadrant du passage à l'échelle (objectif minimal vs avancé, cf. Required Inputs §7) selon le budget calcul réellement disponible.
 5. **Jour 5 — Entraînement principal** 5.1. Lancer l'entraînement retenu (durée/volumétrie décidée en Jour 4) avec checkpointing régulier. 5.2. Suivre le monitoring en continu, générer des visualisations qualitatives à intervalles réguliers.
 6. **Jour 6 (marge) — Consolidation** 6.1. Sélectionner le meilleur checkpoint selon la stabilité de la loss et la qualité visuelle des poses. 6.2. Sauvegarder ce checkpoint hors Git avec documentation de son origine (hyperparamètres, volumétrie, durée d'entraînement). 6.3. Rédiger une note de décision documentant le choix final objectif minimal/avancé, à inclure dans le rapport.
- **Dépendances** : Sprint 5 terminé, GPU disponible (Required Inputs §1).
 - **Critères d'acceptation** : au moins un entraînement stable (loss ne diverge pas) sur 5-10k frames avec poses visuellement plausibles sur un échantillon.
 - **Tests requis** : test de reprise sur checkpoint.
 - **Risques** : R3 (calcul insuffisant), R5 (instabilité).
 - **Definition of Done** : checkpoint stable identifié et sauvegardé, décision documentée sur le passage à l'échelle (objectif minimal vs avancé confirmé).
 - **Effort estimé** : 6 jours.

Sprint 7 (Semaine 7) — Évaluation

- **Objectifs** : évaluation qualitative complète + annotation manuelle + PCK@0.05 (si objectif avancé confirmé).
- **Livrables** : rapport d'évaluation intermédiaire, sous-ensemble Weizmann annoté, script `evaluation/pck.py`.
- **User stories** : US6, US7, US8.
- **Étapes détaillées** :
 1. **Jour 1 — Inférence et visualisation qualitative** 1.1. Écrire `evaluation/inference.py` : chargement du checkpoint final, inférence sur

l'ensemble de Weizmann Horse. 1.2. Générer les visualisations qualitatives (poses 2D superposées + poses 3D) via `evaluation/visualize.py`. 1.3. Revue visuelle systématique avec l'encadrant sur un échantillon représentatif (bonnes prédictions et échecs).

2. **Jour 2 — Pilote d'annotation** 2.1. Choisir/mettre en place l'outil d'annotation (Required Inputs §4). 2.2. Annoter un petit lot pilote (~10 images) pour calibrer la procédure (définition précise des points clés à cliquer, cohérence avec le schéma défini en Sprint 3). 2.3. Ajuster les consignes d'annotation si des ambiguïtés apparaissent.
3. **Jour 3-4 — Annotation définitive** 3.1. Annoter 50 à 100 images Weizmann Horse (borne selon le temps disponible et le niveau d'objectif retenu). 3.2. Effectuer un contrôle qualité croisé sur un sous-échantillon (relecture des annotations).
4. **Jour 5 — Calcul PCK@0.05 et comparaison baseline** 5.1. Implémenter `evaluation/pck.py` (calcul du PCK@0.05 selon la définition standard : seuil = 5% de la taille de la bounding box). 5.2. Calculer le score sur le sous-ensemble annoté et documenter la méthodologie exacte utilisée. 5.3. Implémenter/exécuter le baseline de comparaison retenu (modèle 2D supervisé simple, ou variante sans perte de cohérence géométrique — F6 désactivée). 5.4. Valider manuellement un échantillon du calcul PCK (vérification croisée sur quelques images).

- **Dépendances** : Sprint 6 (checkpoint stable disponible).
- **Critères d'acceptation** : PCK@0.05 calculé et documenté (même si score modeste — l'important est la validité méthodologique), comparaison baseline présente si objectif avancé retenu.
- **Tests requis** : validation croisée manuelle d'un échantillon du calcul PCK.
- **Risques** : R6 (annotation chronophage/incohérente).
- **Definition of Done** : résultats quantitatifs et qualitatifs consolidés.
- **Effort estimé** : 5 jours.

Sprint 8 (Semaine 8) — Analyse, documentation et livraison

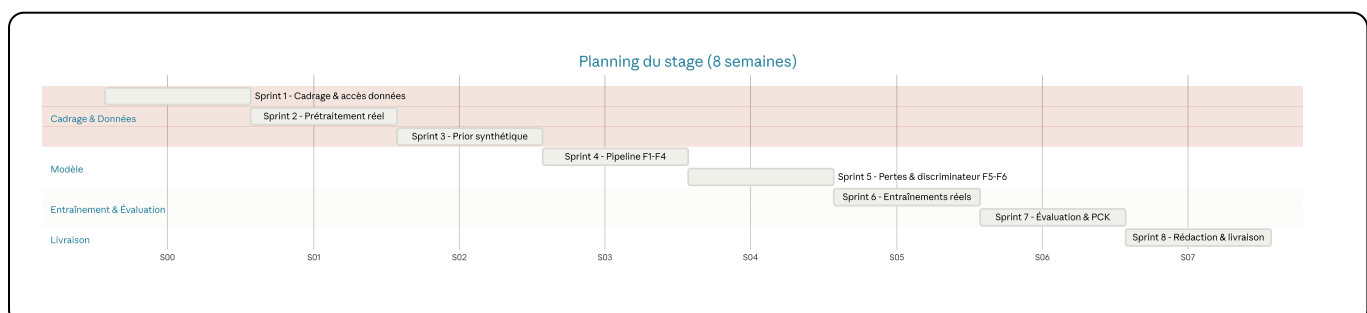
- **Objectifs** : analyse des résultats vs papier, rédaction rapport, préparation démo, dépôt du code documenté.
- **Livrables finaux** : rapport scientifique court, dépôt Git documenté, notebook de visualisation, fiche datasets/licences, slides (10-12).
- **User stories** : US9, US10.
- **Étapes détaillées** :
 1. **Jour 1 — Analyse des résultats** 1.1. Comparer qualitativement et quantitativement les résultats obtenus à ceux annoncés dans le papier original. 1.2. Analyser les écarts (volumétrie réduite, simplifications d'architecture) et leur

impact probable sur les scores. 1.3. Lister explicitement les limites de la reproduction (section dédiée du rapport).

2. **Jour 2 — Rédaction du rapport scientifique** 2.1. Rédiger les sections : contexte, méthode reproduite, données utilisées, résultats, limites (cf. §10 Plan de documentation). 2.2. Intégrer les visualisations qualitatives et le score PCK@0.05 obtenu. 2.3. Relecture croisée avec l'encadrant (planifier ce point tôt dans la semaine pour garder du temps de correction).
3. **Jour 3 — Nettoyage et finalisation du dépôt** 3.1. Nettoyer le code (suppression de scripts expérimentaux non finaux, cohérence des noms de fichiers). 3.2. Finaliser le README (installation, structure, comment relancer chaque étape du pipeline). 3.3. Finaliser la fiche de vérification des datasets et licences (§5.4, tous datasets couverts). 3.4. Rédiger le runbook (`docs/runbook.md`) : comment reprendre un entraînement, relancer l'évaluation.
4. **Jour 4 — Notebook de démonstration et test d'installation propre** 4.1. Finaliser `notebooks/visualization_demo.ipynb` (démonstration interactive des poses 2D/3D). 4.2. Faire tester l'installation du dépôt par l'encadrant (ou un tiers) en suivant uniquement le README, sur un environnement vierge. 4.3. Corriger les étapes manquantes ou ambiguës identifiées lors de ce test.
5. **Jour 5 — Slides et livraison finale** 5.1. Préparer les 10-12 slides de présentation finale (contexte, méthode, résultats, limites, démonstration). 5.2. Répétition de la présentation. 5.3. Taguer la version finale du dépôt Git (tag de release). 5.4. Revue finale et soutenance avec l'encadrant ; validation de la checklist complète (§11).

- **Dépendances** : tous les sprints précédents.
- **Critères d'acceptation** : tous les livrables listés dans le sujet original sont présents et validés (cf. checklist §11).
- **Tests requis** : test d'installation "propre" du dépôt par une tierce personne (encadrant) en suivant uniquement le README.
- **Risques** : R7 (sous-estimation du temps de rédaction).
- **Definition of Done** : soutenance/présentation finale effectuée, dépôt figé (tag de version).
- **Effort estimé** : 5 jours.

8.1 Vue Gantt synthétique



(Dates indicatives — à recalculer sur la date réelle de début de stage.)

9. Registre des risques

ID	Risque	Catégorie	Probabilité	Impact	Mitigation	Plan de contingence
R1	Accès TigDog bloqué ou incomplet	Technique/Data	Moyenne	Élevé	Valider l'accès dès J1, contacter les auteurs si besoin	Basculer sous-ensembles alternatifs vidéos + Weizmann uniquement
R2	Extraction vidéo plus longue que prévu	Opérationnel	Élevée	Moyen	Paralléliser, limiter le nombre de vidéos dès le départ	Réduire volume (objectif minimal)
R3	Dépôt tiers (<i>Learning from Synthetic Animals</i>) non maintenu / cassé	Technique	Moyenne	Élevé	Forker/miroir local dès Semaine 3	Génération manuelle simplifiée prior synthétique réduit
R4	Sous-spécification du module de lifting 2D→3D dans le papier	Technique	Moyenne	Moyen	Documenter explicitement les choix d'interprétation dans le rapport	Simplifier une paramétrisation d'angles articulaires basique
R5	Instabilité d'entraînement GAN (mode collapse, NaN)	Technique	Élevée	Élevé	Monitoring serré, learning rate conservateur, checkpointing fréquent	Réduire pondération la perte adversaire revenir à entraînement en deux (pré-entraînement F1-F4 puis GAN)

ID	Risque	Catégorie	Probabilité	Impact	Mitigation	Plan de contingence
R6	Annotation manuelle Weizmann chronophage/incohérente	Opérationnel	Moyenne	Moyen	Pilote de calibration avant annotation définitive	Réduire images (basse du si nécess
R7	Sous-estimation du temps de rédaction finale	Opérationnel	Moyenne	Moyen	Commencer la rédaction du rapport dès Semaine 5 (au fil de l'eau)	Prioriser rapport sur les sl arbitrage nécessai
R8	Budget calcul GPU insuffisant	Infrastructure	Moyenne-Élevée	Élevé	Clarifier dès Semaine 1 (Required Inputs §1)	Repli exp sur l'obje minimal docume: justifié d rapport
R9	Questions de licence sur les vidéos YouTube extraites	Conformité	Faible-Moyenne	Moyen	Documenter l'usage strictement académique, ne pas republier les vidéos	Utiliser exclusiv TigDog s

10. Plan de documentation

Document	Contenu	Public	Quand
README.md	Installation, structure du dépôt, comment relancer chaque étape	Développeurs / relecteurs futurs	Continu, finalisé Semaine 8
Rapport scientifique court	Papier résumé, méthode reproduite, données, limites, résultats	Encadrant / évaluateurs	Rédaction continue dès S5, finalisé S8
Fiche datasets/licences	Provenance, licence, usage autorisé pour chaque source de données	Conformité / encadrant	Semaine 2-3, finalisée S8

Document	Contenu	Public	Quand
Runbook	Comment relancer un entraînement, reprendre sur checkpoint, relancer l'évaluation	Futur repreneur du projet	Semaine 6-8
Notebook de visualisation	Démonstration interactive des poses 2D/3D prédites	Encadrant / démonstration	Semaine 7-8
Slides finales (10-12)	Synthèse visuelle du travail et des résultats	Jury / encadrant	Semaine 8
Documentation d'architecture (ce document)	Vue d'ensemble technique et méthodologique	Encadrant / stagiaire	Semaine 1, mis à jour en continu

11. Checklist finale de préparation à la livraison

Fonctionnalité

- ☐ Pipeline F1→F6 exécutable de bout en bout sans erreur
- ☐ Entraînement reproductible via un unique script/commande documentée
- ☐ Visualisations qualitatives 2D et 3D générées

Données et conformité

- ☐ Fiche de vérification des datasets et licences complétée
- ☐ Aucune donnée brute volumineuse committée dans Git
- ☐ Usage des vidéos YouTube documenté comme strictement académique

Évaluation

- ☐ Évaluation qualitative réalisée sur Weizmann Horse
- ☐ PCK@0.05 calculé si objectif avancé retenu (sinon justification documentée)
- ☐ Comparaison baseline effectuée si objectif avancé retenu

Qualité et tests

- ☐ Tests unitaires sur modules F1-F6 exécutés et passants
- ☐ Test d'intégration bout-en-bout passant
- ☐ Test de reprise sur checkpoint validé

Documentation

- ☐ README complet et testé sur environnement propre

- ☐ Rapport scientifique court finalisé
- ☐ Runbook de reprise/relance rédigé
- ☐ Notebook de visualisation fonctionnel

Livraison

- ☐ Dépôt Git taggé (version finale du stage)
- ☐ Slides finales (10-12) prêtes
- ☐ Checkpoints finaux sauvegardés hors Git avec lien d'accès documenté
- ☐ Revue finale effectuée avec l'encadrant

(Les items "Reliability/Scalability/Rollback/Disaster recovery" du gabarit générique ne s'appliquent pas à un livrable de recherche non déployé ; ils sont volontairement omis plutôt que remplis artificiellement.)

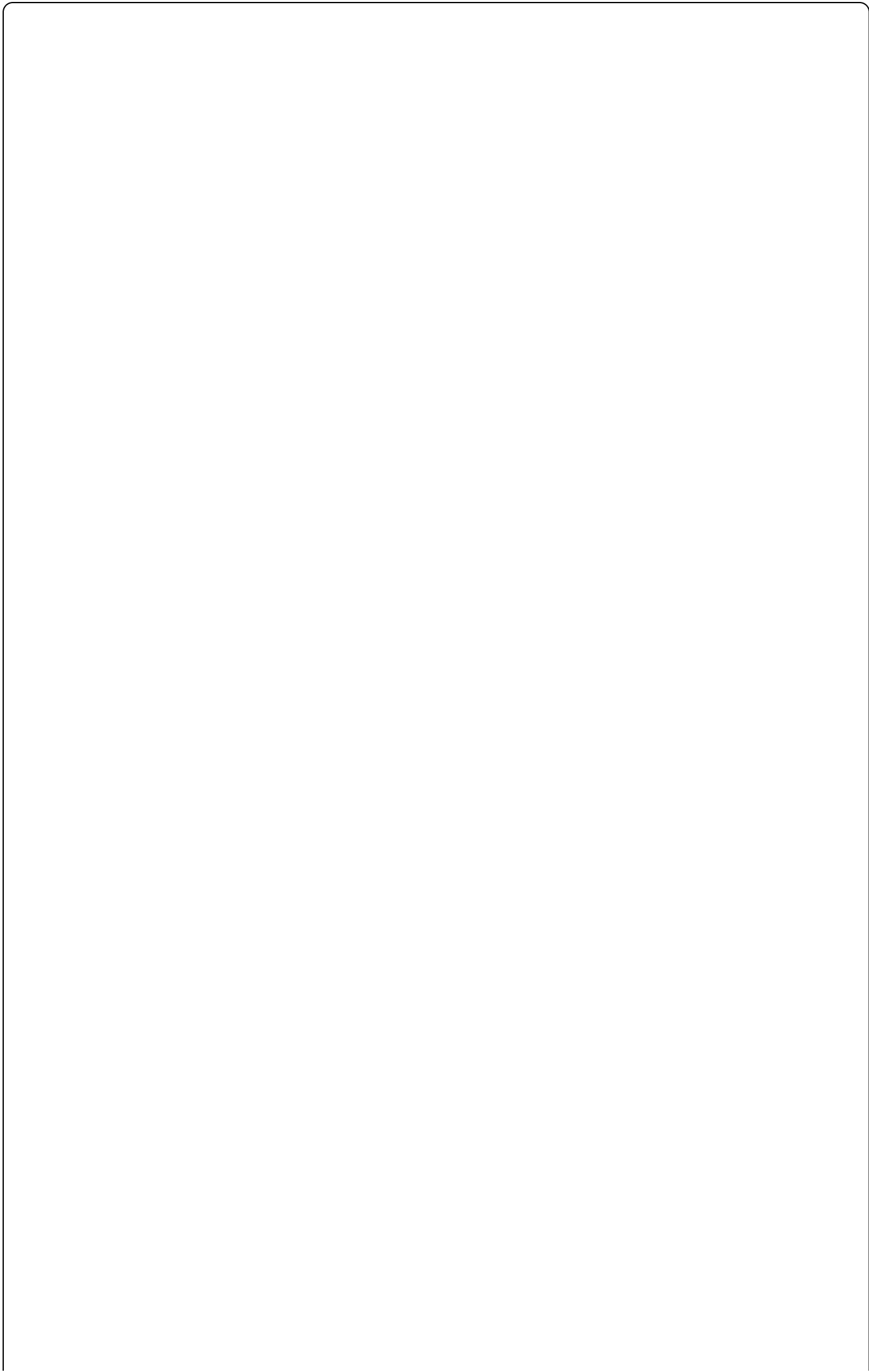
12. Bibliothèque de prompts réutilisables (sans chaîne de raisonnement exposée)

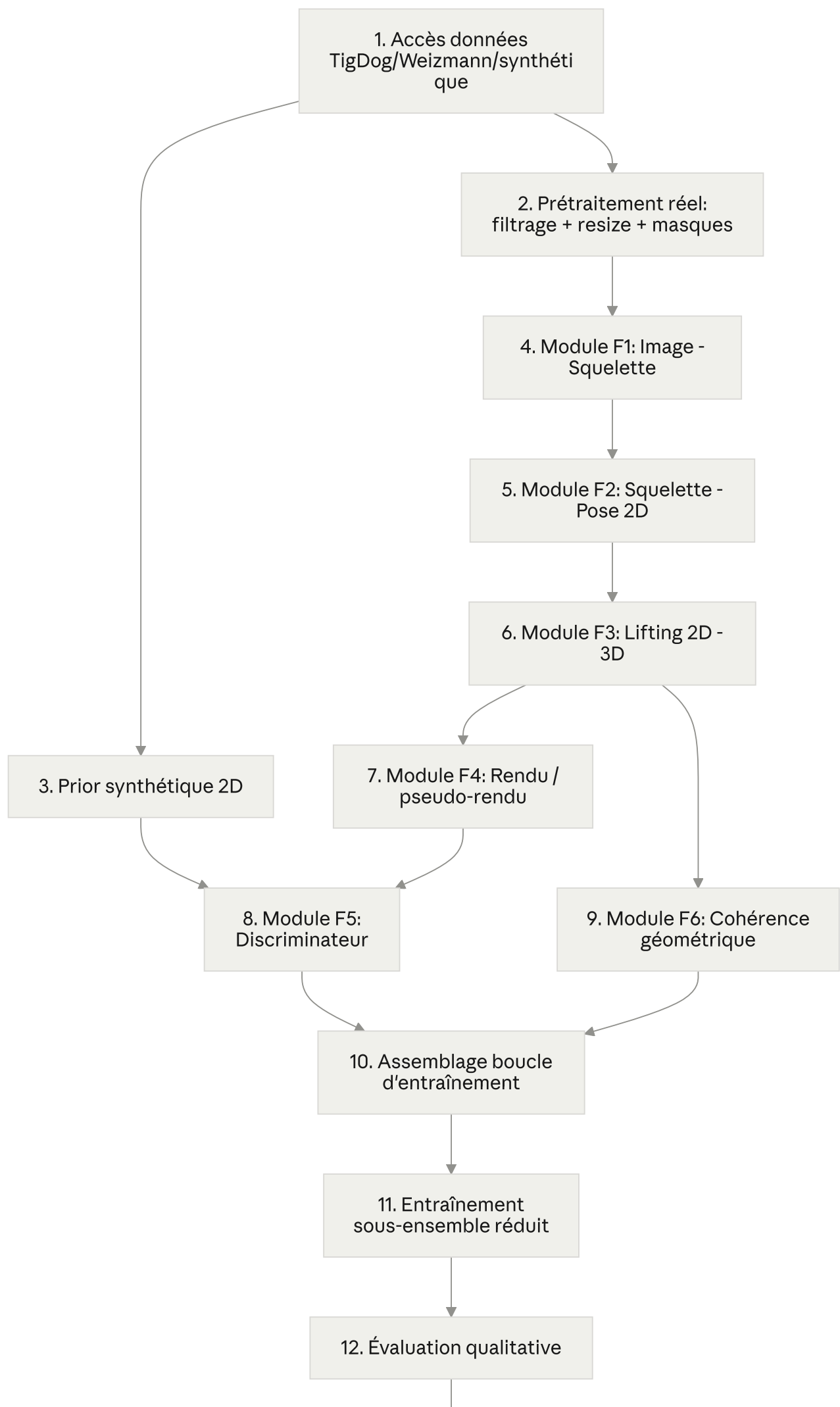
Ces prompts sont conçus pour guider un assistant IA sur des tâches précises **sans** demander ni exposer de raisonnement interne — uniquement des instructions orientées tâche et résultat attendu.

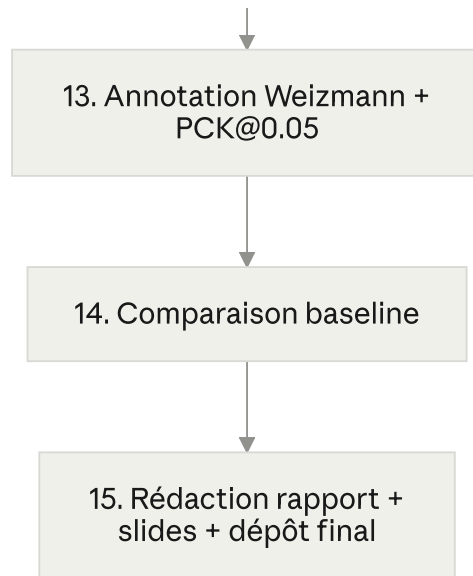
Tâche	Prompt réutilisable
Analyse des besoins	"Résume en 10 points les exigences méthodologiques du papier [titre/liens] pertinentes pour une reproduction simplifiée en 8 semaines. Liste séparément ce qui est indispensable et ce qui peut être simplifié."
Conception d'architecture	"Propose une architecture modulaire PyTorch pour un pipeline composé des étapes suivantes : [lister F1-F6]. Fournis les interfaces (entrées/sorties) de chaque module sous forme de tableau."
Décomposition fonctionnelle	"Découpe le pipeline suivant en modules indépendants testables : [description]. Pour chaque module, indique ses dépendances amont/aval."
Conception de dataset	"Propose une structure de dossiers et un schéma de nommage pour un dataset composé de : images réelles non annotées, masques de segmentation, poses synthétiques 2D. Le format doit être compatible <code>torch.utils.data.Dataset</code> ."
Conception d'API interne	"Définis la signature Python (types d'entrée/sortie) de chaque module du pipeline suivant, sans implémentation : [liste des modules]."
Génération de visualisation	"Écris un script de visualisation qui superpose un squelette 2D prédit sur l'image source, à partir d'un tenseur de coordonnées de points clés au format [format]."
Implémentation backend/pipeline	"Implémente en PyTorch le module suivant : [nom du module], avec cette entrée [forme du tenseur] et cette sortie attendue [forme du tenseur]. N'ajoute aucune dépendance externe non listée."
Planification de validation	"Liste les vérifications à effectuer avant de lancer un entraînement long sur le pipeline suivant : [description], en couvrant données, configuration et stabilité numérique."
Génération de tests	"Écris des tests unitaires PyTest pour le module [nom], vérifiant les formes de tenseurs en entrée/sortie et le comportement sur des cas limites : [lister les cas limites]."
Revue de sécurité/conformité	"Vérifie si le script suivant expose des secrets, des chemins codés en dur, ou des données non filtrées avant commit : [coller le script]."
Optimisation de performance	"Analyse ce script d'entraînement PyTorch et identifie les goulots d'étranglement probables (I/O, transferts CPU/GPU, taille de batch) sans modifier la logique métier : [coller le script]."

Tâche	Prompt réutilisable
Refactoring	"Réorganise ce module pour séparer la logique de données, la logique de modèle et la logique d'entraînement, sans changer le comportement observable : [coller le code]."
Génération de documentation	"Rédige la section README correspondant à l'installation et au lancement de ce script : [coller le script], en listant les prérequis explicitement."
Revue de code	"Relis ce module PyTorch et liste les problèmes potentiels de forme de tenseur, de fuite de mémoire GPU, ou de non-déterminisme : [coller le code]."
Investigation de bug	"Voici une erreur obtenue lors de l'exécution de [script] : [message d'erreur complet]. Liste les causes possibles par ordre de probabilité, sans supposer d'informations non fournies."
Planification de sprint	"À partir de cette liste de tâches restantes [liste], propose un découpage en sprints d'une semaine avec critères d'acceptation explicites pour chacun."
Planification de release/livraison	"Vérifie que les livrables suivants sont complets et cohérents entre eux avant dépôt final : [lister les livrables]."
Vérification de déploiement/livraison	"Simule l'installation de ce dépôt sur un environnement vierge à partir du seul README, et signale toute étape manquante ou ambiguë : [coller le README]."

13. Ordre d'implémentation optimal (dépendances)







Justification de l'ordre :

1. **Les données précèdent tout** : aucun module modèle ne peut être testé sérieusement sans données réelles et sans prior synthétique — les développer en parallèle (branches A→B et A→C) est possible et recommandé pour gagner du temps (Sprints 2 et 3 peuvent se chevaucher partiellement si les ressources le permettent).
2. **F1→F2→F3→F4 est une chaîne séquentielle stricte** : chaque module consomme la sortie du précédent, donc pas de parallélisation possible sur cette branche.
3. **F5 dépend à la fois du prior synthétique (C) et de la sortie du pipeline (G)** : ne peut être implémenté avant que les deux existent.
4. **F6 dépend uniquement de F3** (la pose 3D), et peut donc être développé en parallèle de F4/F5 une fois F3 disponible — opportunité de parallélisation en Sprint 5.
5. **L'entraînement (J-K) ne peut démarrer qu'une fois tous les modules et pertes assemblés** — c'est le point de convergence obligatoire de toutes les branches.
6. **L'évaluation et la rédaction sont strictement postérieures** à l'obtention d'au moins un checkpoint stable.

Synthèse des écarts assumés par rapport au gabarit générique de plan d'implémentation

Ce document adapte délibérément un gabarit générique de "plan d'implémentation logiciel" à un **projet de recherche reproductible** de 2 mois. Certaines sections du gabarit d'origine (authentification multi-utilisateur, API publique, accessibilité UI, localisation, haute disponibilité, plan de reprise après sinistre en production) ont été volontairement omises ou réduites à leur équivalent pertinent, conformément à l'instruction de ne pas fabriquer de détails non pertinents pour le projet réel. Les zones encore incertaines sont listées en section 0 (*Required Inputs*) et doivent être tranchées avec l'encadrant en semaine 1.